

Design Documentation

SkillSwap — a skill-exchange web app

Project Title

SkillSwap — A Peer Skill-Exchange Platform

Participant Information

- **Name:** Eva Alex
- **Age / Category:** 13 / Middle (11–13)

Theme Analysis

Anyone can list what they're able to teach and what they'd like to learn, and get matched with people whose skills complement theirs. Every user is both a potential teacher and learner — the core idea of a "swap" rather than one-directional tutoring.

Problem Statement

People with skills to share don't know who wants to learn them, and people who want to learn something new have no easy informal way to find a teacher. SkillSwap matches people on complementary skills and gives them a simple way to connect and chat.

Target Users

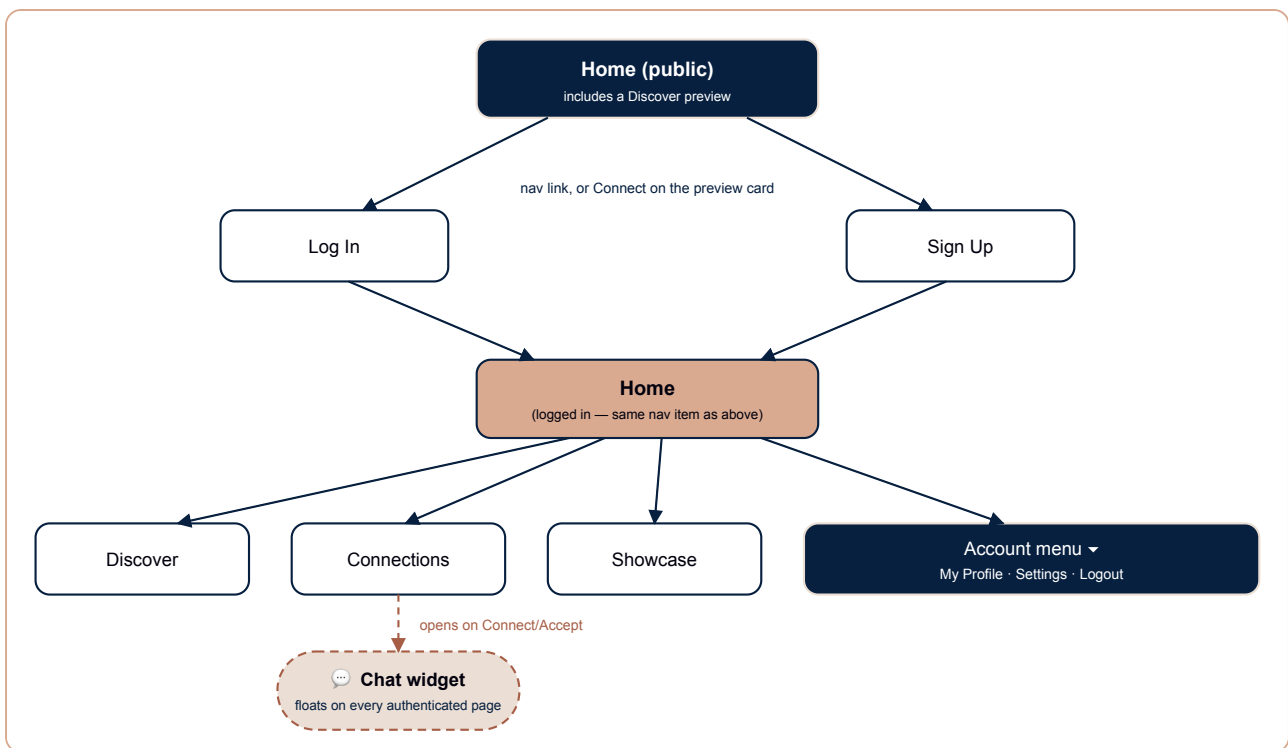
Students and hobbyists who'd rather trade skills informally than pay for lessons — e.g. someone who can teach guitar and wants to learn to code. The 13-category system (Technology, Art, Music, Languages, Sports, Academics, Cooking, Business, Wellness, Crafts, Writing, Games, Other) targets a general, mostly student-age audience.

Proposed Solution

Every profile lists skills to **teach** and skills to **learn**, each tagged with a category. **Discover** ranks other users by a percent match score with a plain-language reason attached to each ("They can teach you Guitar"). **Connect** sends a request; the other person **Accepts** or **Rejects** from **Connections** — chat only unlocks after acceptance, via a floating widget that supports text and media attachments. **Showcase** lets users post a photo or video with a description, visible to everyone.

Information Architecture

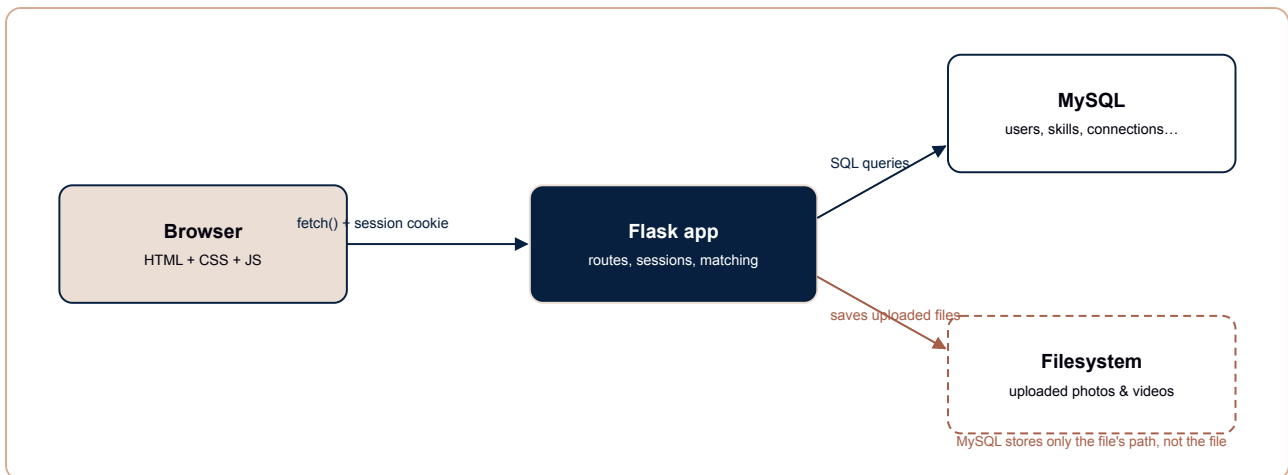
One public entry point (**Home**) becomes the logged-in dashboard after sign-up/login — there's no separate "Dashboard" page, just a nav item that always says "Home." Four sections hang off the main nav: Discover, Connections, Showcase, and an Account dropdown (My Profile, Settings, Logout) kept out of the primary nav.



Design Decisions

- **Single-role profiles**, not separate teacher/student accounts — a "swap" means most people are both.
- **Percent match score with reasons**, not a bare number — "They can teach you Guitar" makes a match trustworthy, not just a ranking.
- **Explicit connect** → **accept/reject**, not instant chat — mirrors how you'd actually agree to meet someone new.
- **Floating chat widget** instead of a full chat page, so conversations stay reachable from anywhere on the site.
- **Fixed categories with an editable keyword suggestion**, rather than free text, so Discover's category filter stays reliable.
- **Color palette** — navy (#07203F) for structure/actions, tan (#D9AA90) for the logged-in state, cream (#EBDED4) backgrounds, rust (#A65E46) as an accent on the chat widget and borders.

Technical Approach



- **Stack:** Python/Flask backend, MySQL database, vanilla HTML/CSS/JS frontend — no framework or build step.
- **Auth:** server-side sessions with salted `script` password hashing. Every route reads "who is this" from the session cookie, never from client-supplied data.
- **Matching algorithm:** teach/learn skill lists are normalized and compared as sets; the score is the overlap divided by total skills listed, capped at 100%, labeled Excellent/Good/Partial/No Overlap Yet, with a reason per overlapping skill.
- **Data model:** `users`, `skills` (teach/learn + category), `connections` (pending/accepted), `messages` (text + attachment, read receipts), `feedback` (rating + comment), `showcase_posts`.
- **Chat delivery is polling** — a deliberate simplicity choice for a project this size.
- **File uploads** are saved to disk under a generated filename; the database stores only the resulting path.

AI Usage Disclosure

Claude (Claude Code) was used throughout — to scaffold the initial Flask/MySQL backend and frontend, to replace an early client-side identity system with real authentication and a proper connect-request/accept/reject flow, and for ongoing feature work and bug fixes. Full details are in the AI Usage Log.